

Transazioni, account, calldata, storage e revert

`msg.sender` · `msg.value` · `msg.data`

Ambiente esclusivamente didattico, difensivo e locale:
contratti giocattolo, account di test, fondi fittizi.

Baseline: Solidity 0.8.37
verificata il 21 settembre 2026

1 · OBIETTIVI

Cosa saprai leggere in una funzione Solidity

01

Da dove nasce l'esecuzione

Transazione top-level, message call o chiamata interna Solidity.

02

Cosa controlla il chiamante

`msg.sender` come chiamante diretto del frame, `msg.value`, i parametri ABI e `msg.data`.

03

Cosa resta e cosa si annulla

`calldata`, `memory` e `storage`; cosa persiste tra una transazione e l'altra; cosa fa il `revert` allo stato.

04

Come lo verifichi

Requisiti trasformati in proprietà e test negativi; Foundry per simulare chiamanti, fondi fittizi e revert attesi.

L'ABITUDINE DA PRENDERE SUBITO

Ogni dato proveniente dal chiamante è non fidato finché il contratto non ne ha verificato le proprietà necessarie.

Non solo i parametri espliciti della funzione: anche `msg.sender`, `msg.value` e, più in generale, il contesto della chiamata.

Ethereum è una macchina a stati globale



Tre concetti da tenere separati

1

Transazione

L'istruzione top-level che entra nel protocollo Ethereum: mittente e autorizzazione, nonce, destinazione, eventuale value, input data, limiti e costi di gas.

2

Message call

Un contratto ne chiama un altro: nasce un nuovo contesto di esecuzione. Lì `msg.sender` è il chiamante diretto, non per forza chi ha originato la transazione.

3

Chiamata interna

Una funzione `internal`, invocata senza una vera external call, non crea lo stesso confine esterno. Conterà per external calls e reentrancy.

Un EOA non è più per forza «senza codice»

IL MODELLO SEMPLIFICATO

EOA senza codice, contract account con codice

Resta utile come prima approssimazione.

DOPO PECTRA ED EIP-7702

Un EOA può delegare l'esecuzione a codice

La distinzione non regge più proprietà di sicurezza rigide.

REGOLA DEL CORSO

Non dedurre capacità o affidabilità di un chiamante soltanto dal fatto che sembri un EOA. Progetta assumendo che un partecipante possa avere comportamento programmabile.

Per questo `tx.origin == msg.sender` non è una protezione generale valida contro reentrancy o «chiamanti contract».

3.1 • LA TRANSAZIONE

Una transazione è un input alla macchina a stati

Campo	Perché conta per la sicurezza
destinazione	quale account o contratto riceve la chiamata
value	quanta ETH viene trasferita con la chiamata top-level
input data	i byte con selector e argomenti ABI-encoded, per una normale chiamata Solidity
nonce	evita che due normali transazioni dello stesso sender abbiano lo stesso numero di sequenza
gas	limita il lavoro computazionale che l'esecuzione può consumare

All'EVM arrivano byte: una transazione non «chiama una funzione per nome». Solidity usa l'ABI per dare struttura a quei byte.

3.2 • ABI E CALldata

Il contratto riceve byte, non nomi

msg.data

byte 0..3

function selector

byte 4 ...

argomenti ABI-encoded

Il selector

I primi quattro byte di keccak256 della signature canonica:

record(address, bytes)

Nome e tipi. Niente nomi delle variabili, niente tipi di ritorno.

Cosa espone Solidity

```
msg.data // bytes calldata: l'intera calldata  
msg.sig  // bytes4: i primi 4 byte di msg.data
```

3.2 · STRUTTURA E SEMANTICA

Il tipo è valido. Il valore ha senso?

```
function record(address beneficiary, bytes calldata note)
    external payable
```

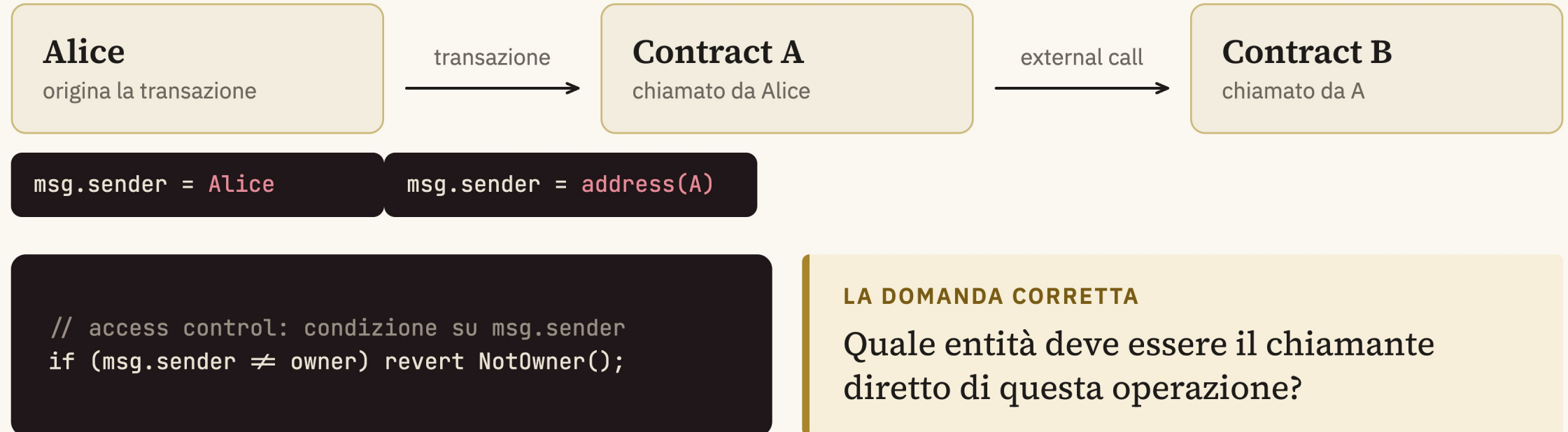
beneficiary e note arrivano dalla calldata: li sceglie il chiamante. L'ABI decoder controlla la forma, non il significato.

Valido per l'ABI	Ma per il requisito?
address valido sintatticamente	address(0) è accettabile?
uint256 valido	zero è permesso? esiste un massimo?
bytes validi	la lunghezza deve essere limitata?
msg.value valido	coincide con quanto la logica si aspetta?

ABI decoding garantisce una certa struttura; la business logic deve garantire la validità semantica.

3.3 • MSG.SENDER

msg.sender è il chiamante diretto del frame



3.3 · TX.ORIGIN

tx.origin non è un sostituto di msg.sender

tx.origin

L'origine della transazione lungo la call chain. Non cambia quando un contratto intermedio fa altre call: per questo è pericoloso come autorizzazione generale.

Dopo EIP-7702 non rappresenta per forza un account «non programmabile».

msg.sender

Il sender del messaggio corrente: cambia lungo la call chain, frame per frame. È il chiamante diretto che la funzione vede davvero.

REGOLA DEL CORSO

Per l'autorizzazione ordinaria ragiona sul chiamante appropriato del frame: msg.sender o ruoli espliciti. Non usare tx.origin come scorciatoia.

3.4 · MSG.VALUE

msg.value: l'ETH della chiamata corrente

msg.value è un uint256 espresso in wei

`1 ether = 10^18 wei`

```
function deposit() external payable {  
    // msg.value contiene i wei ricevuti con questa call  
}  
  
if (msg.value == 0) revert ZeroValue();
```

payable, o fallisce

Senza payable, una chiamata public o external che tenta di inviare ETH fallisce.

Lo sceglie il chiamante

Entro il proprio saldo e la transazione: il contratto verifica, non presume il valore giusto.

Appartiene al call frame

In una call verso un altro contratto si allega un valore: il callee lo vede come suo msg.value.

calldata, memory, storage: tre aree diverse

calldata

Input read-only della call corrente.

PER LA SICUREZZA

Il confine di ingresso di dati controllabili dall'esterno. Domanda d'audit: quale proprietà sto assumendo senza verificarla?

memory

Area temporanea della call corrente.

PER LA SICUREZZA

Per i parametri dinamici di funzioni external, calldata evita di copiare qui l'input senza necessità.

storage

Stato persistente associato al contratto.

PER LA SICUREZZA

Persiste tra una transazione e la successiva. Slot da 32 byte; mapping e array dinamici localizzati tramite hashing.

Una scrittura in storage cambia la fonte di verità persistente del protocollo.

Cinque domande per ogni funzione che scrive

- 1 Quale storage legge?
- 2 Quale storage modifica?
- 3 Quale relazione tra variabili deve rimanere vera dopo la modifica?
- 4 Chi può causare quella modifica?
- 5 Quali input controlla il chiamante?

Queste domande diventeranno il nucleo della nostra metodologia di audit.

3.6 • VALORI DI DEFAULT

Il valore di default conta

```
uint256 → 0  
address → address(0)  
bool → false  
bytes32 → 0x00...00
```

Non inizializzato o legittimo?

Uno stato mai scritto restituisce il default: se non lo consideri, si confonde con un valore valido.

Un esempio

Se `id == 0` è un ID valido, `deposits[id].amount == 0` non basta a dire in ogni design che il deposito non esiste.

NEL CONTRATTO DI OGGI

Un deposito esiste se e solo se `id < nextId`: un criterio esplicito, non un default.

3.7 · REVERT E ATOMICITÀ

Quello che il revert annulla, non resta

```
nextId = nextId + 1;  
  
if (badCondition) {  
    revert BadCondition();  
}
```

Se il revert annulla l'intera call, l'incremento di `nextId` non rimane nello stato persistente: le modifiche del contesto annullato non vengono committate.

LA DOMANDA DI OGNI TEST NEGATIVO

Dopo il fallimento, lo stato è davvero rimasto invariato nelle parti che dovevano essere atomiche?

Revert non significa gratuito. Annulla gli effetti di stato, ma il lavoro computazionale già eseguito consuma gas.

3.7 · FALLIRE IN MODO ESPPLICITO

require, revert, assert: usi diversi

```
require(condition);  
require(condition, "reason");  
revert CustomError(...);  
assert(internalInvariant);
```

LO STILE DEL CORSO

```
error ZeroValue();  
  
if (msg.value == 0) revert ZeroValue();
```

require o revert con custom error

Per condizioni su input, autorizzazione o componenti esterni.

assert

Per proprietà interne impossibili da violare se il codice è corretto. Mai al posto di una validazione dell'utente.

Custom errors: espliciti, strutturati e normalmente più efficienti delle lunghe revert string.

3.8 · IL PRIMO THREAT MODEL

Cosa controlla un chiamante ostile di record?

Il chiamante sceglie	Che cosa
<code>msg.sender</code>	l'identità del chiamante diretto, compatibile con il flusso di call
<code>msg.value</code>	l'ETH allegata alla call
<code>beneficiary</code>	l'address passato come input
<code>note</code>	bytes arbitrari, entro i limiti pratici della transazione
<code>momento e ordine</code>	quando chiamare, e in quale sequenza rispetto ad altre chiamate

Il contratto controlla le regole con cui questi input diventano stato persistente.

La sicurezza applicativa non impedisce gli input ostili: garantisce che nemmeno input ostili possano rompere le proprietà dichiarate.

Un Escrow, un pezzo alla volta

Oggi solo il registro locale dei depositi: abbastanza per esercitare `msg.sender`, `msg.value`, `calldata`, `storage` e `revert`.

NON ANCORA

`release`, `refund`, chiamate esterne: sarebbe troppo presto.

CONTRATTO GIOCATTOLO

Non production-ready: mai usarlo per custodire fondi reali.

Il contratto: errori, struct, stato, evento

```
// pragma solidity 0.8.37;
uint256 public constant MAX_NOTE_BYTES = 256;

error ZeroBeneficiary();
error ZeroValue();
error NoteTooLong(uint256 actualLength);
error UnknownDeposit(uint256 id);

struct Deposit {
    address payer;
    address beneficiary;
    uint256 amount;
    bytes32 noteHash;
}
```

```
// stato persistente ed evento
uint256 public nextId;
uint256 public totalRecorded;

mapping(uint256 ⇒ Deposit) private _deposits;
mapping(address ⇒ uint256) public credited;

event DepositRecorded(
    uint256 indexed id,
    address indexed payer,
    address indexed beneficiary,
    uint256 amount,
    bytes32 noteHash
);
```

Quattro variabili in storage, quattro errori custom: ogni rifiuto ha un nome, ogni scrittura una fonte di verità.

4 · LA FUNZIONE CHE SCRIVE

record: prima i check, poi lo storage

```
function record(address beneficiary, bytes calldata note)
    external payable returns (uint256 id)
{
    if (beneficiary == address(0)) revert ZeroBeneficiary();
    if (msg.value == 0) revert ZeroValue();
    if (note.length > MAX_NOTE_BYTES) revert NoteTooLong(note.length);
    id = nextId;
    nextId = id + 1;
    bytes32 noteHash = keccak256(note);
    _deposits[id] = Deposit({
        payer: msg.sender, beneficiary: beneficiary,
        amount: msg.value, noteHash: noteHash
    });
    credited[beneficiary] += msg.value;
    totalRecorded += msg.value;
    emit DepositRecorded(id, msg.sender,
        beneficiary, msg.value, noteHash);
}
```

1 · TRE CHECK

In testa, prima di qualunque scrittura: beneficiary, valore, lunghezza della nota.

2 · SCRITTURE

nextId, _deposits[id], credited, totalRecorded.

3 · EVENTO

Nessuna chiamata esterna nel corpo della funzione.

Perché salviamo l'hash della nota, e con un limite

`keccak256(note)`

`note` è input del chiamante, e nessun requisito chiede di conservarla on-chain.

L'hash permette di verificare in seguito che una nota corrisponda all'originale, senza uno storage proporzionale alla sua dimensione.

`MAX_NOTE_BYTES = 256`

Un input dinamico senza bound è una superficie di costo e di complessità.

Oltre il limite, record reverte con `NoteTooLong(note.length)`.

NON ESISTE UN NUMERO UNIVERSALMENTE SICURO

256 è una scelta didattica: in un sistema reale il bound deve derivare dal requisito.

5.1 • ANALISI DEL CODICE

record letta come la legge un auditor

Domanda	Risposta
Chi può chiamarla?	Chiunque: nessun access control, per requisito del giocattolo
Input del chiamante	beneficiary, note, msg.value, il proprio ruolo di msg.sender, ordine e momento
Valore in ingresso	msg.value wei; lo zero è rifiutato
Stato letto	nextId; credited[beneficiary] e totalRecorded per gli incrementi
Stato scritto	nextId, _deposits[id], credited[beneficiary], totalRecorded, e cresce il saldo ETH del contratto
Chiamate esterne	Nessuna: il controllo non passa mai a codice esterno

Oggi manca il problema «ho aggiornato metà stato, poi cedo il controllo a un destinatario arbitrario»: arriverà con le external calls.

Ogni assunzione ha un custode

DIVENTATE CHECK

- `address(0)` non è un beneficiary valido
- un deposito ha valore positivo
- la nota è libera nel contenuto, ma non oltre 256 byte

SCELTE DI DESIGN

- il payer è il chiamante diretto: si registra `msg.sender`
- non serve conservare la nota, basta il suo hash

GARANTITA DAL LINGUAGGIO

L'aritmetica checked di Solidity 0.8.x, fuori dai blocchi unchecked, evita il wrap silenzioso degli incrementi.

Un auditor distingue l'assunzione documentata e verificata dall'assunzione implicita non protetta.

getDeposit e i getter automatici

```
function getDeposit(uint256 id)
    external view
    returns (Deposit memory deposit_)
{
    if (id ≥ nextId) revert UnknownDeposit(id);
    deposit_ = _deposits[id];
}
```

```
uint256 public nextId;
uint256 public totalRecorded;
mapping(address ⇒ uint256) public credited;
```

Esiste se `id < nextId`

Vale perché gli ID nascono consecutivi da zero e non si cancella nulla. Con cancellazioni o ID non consecutivi va rivalutato.

public non significa «chiunque può modificare»: il compilatore genera un getter di sola lettura. La mutabilità dipende dalle funzioni che scrivono lo storage.

6 • PROPRIETÀ E INVARIANTI

Dieci frasi falsificabili, prima dei test

ID	Proprietà	Enunciato
P1	Niente valore zero	Se <code>msg.value == 0</code> , record reverte e non crea alcun deposito
P2	Niente beneficiary nullo	Se <code>beneficiary == address(0)</code> , record reverte e non tocca la contabilità
P3	Bound sulla calldata	Se <code>note.length > 256</code> , record reverte
P4	Identità del payer	Dopo un successo, <code>deposit.payer</code> è esattamente il <code>msg.sender</code> della call
P5	Valore registrato	Dopo un successo, <code>deposit.amount</code> è esattamente il <code>msg.value</code> della call
P6	Progressione degli ID	Un successo incrementa <code>nextId</code> esattamente di 1; un fallimento no
P7	Contabilità del beneficiary	Un successo aumenta <code>credited[beneficiary]</code> esattamente di <code>msg.value</code>
P8	Totale contabilizzato	Un successo aumenta <code>totalRecorded</code> di <code>msg.value</code> ; un fallimento non lo modifica
P9	Atomicità del fallimento	Se record reverte, nessuna scrittura contabile resta e l'ETH della call non resta nel contratto
P10	Hash della nota	Per un deposito riuscito, <code>noteHash == keccak256(note)</code>

6 · ATTENZIONE

Una quasi-invariante da non formulare male

LA TENTAZIONE

`address(escrow).balance = totalRecorded`

Nel laboratorio sembrerà vera. Come proprietà generale è fragile: un contratto non può sempre impedire che gli venga accreditata ETH per vie che non passano da record.

Saldo osservato

Quanta ETH ha il contratto, da qualunque via sia arrivata.

Accounting interno

Quanto il protocollo ha registrato lungo il percorso applicativo previsto.

Più prudente: ragionare sulla relazione tra contabilità interna e fondi necessari a soddisfarla. Questa distinzione tornerà spesso nel corso.

Installa, crea il progetto, pinna il compilatore

```
# installa con Foundryup
curl -L https://getfoundry.sh/install | bash && foundryup
forge --version
cast --version
anvil --version

# crea il progetto
forge init solidity-security-course
cd solidity-security-course
```

```
# foundry.toml
[profile.default]
src = "src"
out = "out"
libs = ["lib"]
solc_version = "0.8.37"
```

```
solidity-security-course/
├── foundry.toml
├── lib/
│   └── forge-std/
├── src/
│   ├── EscrowLesson1.sol
│   └── OriginAuthToy.sol
└── test/
    ├── EscrowLesson1.t.sol
    └── OriginAuthToy.t.sol
```

Perché il pin: una release futura del compilatore non cambia in silenzio l'ambiente della lezione. Controlla con forge config.

7.5 · IL TEST PRINCIPALE

Dal chiamante allo storage, in un test

```
// test/EscrowLesson1.t.sol (estratto)
function setUp() public {
    escrow = new EscrowLesson1();
    vm.deal(alice, 10 ether);
}

function test_RecordBindsSenderValueAndCalldataToStorage() public {
    bytes memory note = bytes("ordine-42");
    vm.prank(alice);
    uint256 id = escrow.record{value: 1 ether}(bob, note);

    assertEq(escrow.nextId(), 1);
    assertEq(escrow.credited(bob), 1 ether);
    EscrowLesson1.Deposit memory deposit_ = escrow.getDeposit(id);
    assertEq(deposit_.payer, alice);
    assertEq(deposit_.noteHash, keccak256(note));
    // ... anche totalRecorded, saldo, beneficiary, amount
}
```

vm.deal

Assegna ETH fittizia a un account di test.

vm.prank

La call successiva ha quell'account come msg.sender.

vm.expectRevert

La call successiva deve revertire con quell'errore.

Esegui i test, poi guarda i byte

```
forge build
forge test
forge test -vvv

forge test \
  --match-test test_RevertRollsBackStateAndIncomingValue \
  -VVVVV
```

Più v, trace più dettagliate: con **-vvvvv** la trace EVM include anche gli storage changes. Il rollback si vede.

```
cast sig "record(address,bytes)"

cast calldata \
  "record(address,bytes)" \
  0x00000000000000000000000000000000000000000000000000000000000000B0B \
  0xf7264696e652d3432
```

Nessuna blockchain serve per il selector. **0xf7264696e652d3432** è «ordine-42» in UTF-8 esadecimale.

`0x[4-byte selector][argomenti ABI...]`

Non memorizzare l'hex: il contratto riceve byte scelti dal chiamante e li interpreta secondo l'ABI.

7.8 · ESTENSIONE FACOLTATIVA

Una vera transazione, su un nodo locale

```
# terminale 1
anvil

# terminale 2: deploy
forge create src/EscrowLesson1.sol:EscrowLesson1 \
  --rpc-url http://127.0.0.1:8545 \
  --private-key <CHIAVE_TEST_STAMPATA_DA_ANVIL>
```

```
# transazione locale
cast send <INDIRIZZO_CONTRATTO> \
  "record(address,bytes)" \
  0x000000000000000000000000000000000000000000000000B0B \
  0x6f7264696e652d3432 \
  --value 1ether \
  --rpc-url http://127.0.0.1:8545 \
  --private-key <CHIAVE_TEST_STAMPATA_DA_ANVIL>

# lettura, senza nuova transazione
cast call <INDIRIZZO_CONTRATTO> \
  "nextId()(uint256)" \
  --rpc-url http://127.0.0.1:8545
```

Risultato atteso: nextId vale 1.

Solo 127.0.0.1: le chiavi di Anvil e i suoi fondi fittizi non escono dal nodo locale.

8 • TEST NEGATIVI

Dopo il revert, controlla lo stato

L'happy path prova che almeno un input valido funziona. Le quattro classi di rifiuto già coperte:

```
msg.value = 0
```

```
beneficiary = address(0)
```

```
note.length > MAX_NOTE_BYTES
```

```
id inesistente in getDeposit
```

NON BASTA

```
vm.expectRevert(...);  
escrow.record(...);
```

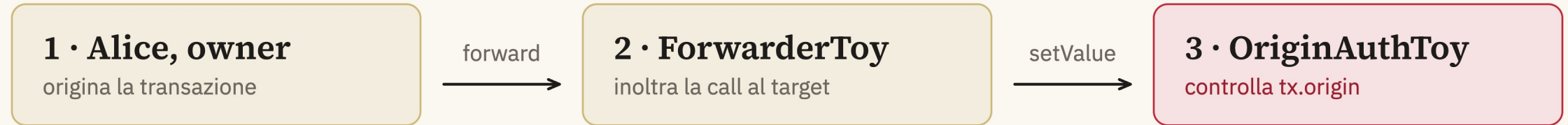
ANCHE LO STATO DOPO IL REVERT

```
assertEq(escrow.nextId(), 0);  
assertEq(escrow.totalRecorded(), 0);  
assertEq(escrow.credited(bob), 0);  
assertEq(address(escrow).balance, 0);
```

Il fallimento corretto non è «è comparso un errore»: è «la transizione di stato proibita non è avvenuta».

9 · ERRORE DI PROGETTAZIONE

Autorizzare con tx.origin: il micro-laboratorio



```
msg.sender = Alice
```

```
tx.origin = Alice  
msg.sender = Forwarder
```

```
// VULNERABILE: autorizza l'origine della tx  
if (tx.origin ≠ owner) revert NotOwner();  
value = newValue;
```

Esito: value diventa 777. Il target accetta una call il cui msg.sender è il forwarder, solo perché la transazione l'ha originata Alice.

Volevamo «solo Alice chiama direttamente setValue»; il codice verifica «la call chain è partita da Alice».

Quale proprietà verifica davvero il check?

PRIMA · ORIGINAUTHTOY

```
if (tx.origin ≠ owner) revert NotOwner();
```

Verifica: «la transazione è stata originata da owner».

DOPO · SENDERAUTHTOY

```
if (msg.sender ≠ owner) revert NotOwner();
```

Verifica: «il chiamante diretto di questa funzione è owner».

Test in test/OriginAuthToy.t.sol	Esito atteso
test_Vulnerable_TxOriginAcceptsIntermediaryCall	via forwarder: value = 777, accettata
test_Fixed_MsgSenderRejectsIntermediaryCall	via forwarder: NotOwner, value = 0
test_Fixed_OwnerCanCallDirectly	owner diretto: value = 777

```
forge test --match-path test/OriginAuthToy.t.sol -vvv
```

È anche un regression test: se tx.origin tornasse nella versione fixed, fallirebbe.

Chi arriva, con cosa, e cosa tocca

1	Caller	Chi può arrivare qui? Il controllo usa msg.sender, ruoli, firme o nessuna autorizzazione?
2	Value	La funzione è payable? Quali valori di msg.value sono validi? Zero è previsto?
3	Calldata	Quali parametri controlla l'esterno? Sono validati semanticamente, non soltanto tipizzati?
4	Storage reads	Quali decisioni dipendono dallo stato corrente?
5	Storage writes	Quali variabili persistenti cambiano? Devono cambiare insieme?

Una sequenza compatta, da ripetere su ogni state-changing function.

Cosa succede se qualcosa va storto

6	Failure	Se una condizione fallisce, il revert avviene prima che lo stato possa restare incoerente?
7	External control	Ci sono external calls? In quale punto il controllo passa fuori? Il nostro Escrow, oggi, non ne ha.
8	Assumptions	Si assume che un address sia EOA, che il saldo equivalga all'accounting, che un input sia «ragionevole»?
9	Negative path	Ogni rifiuto importante ha un test che controlla anche lo stato dopo il revert?
10	Property	Sai dire in una frase verificabile cosa resta vero dopo ogni successo e ogni fallimento?

Una voce senza risposta è un punto da investigare, non necessariamente una vulnerabilità.

Prima formula la proprietà, poi scrivi il codice

1 · Due depositi, stesso beneficiary

Alice 1 ether per Bob, un altro account 2 ether: `nextId == 2`, `credited[bob] == 3` ether, payer diversi.

2 · Due beneficiary diversi

Bob e Carol: il credito di Carol non altera quello di Bob. Quale relazione tra mapping key verifichi?

3 · Boundary della nota

`note.length == 256` deve riuscire, 257 deve revertire: un classico boundary test.

4 · Atomicità su stato esistente

Due depositi validi, poi un terzo invalido: nulla cambia, depositi precedenti inclusi.

5 · Analisi prima del test

Senza eseguire codice, compila la tabella di record, poi confrontala con la sezione 5.

6 · Calldata con Cast

Genera calldata con un'altra nota: trova selector, address codificato, head e tail del bytes.

7 · Threat modeling di un design

Senza il check su `MAX_NOTE_BYTES`: cosa diventa non bounded? Quali costi crescono?

8 · `msg.sender` e intermediario

ForwarderToy emette un evento; nelle trace annota `msg.sender` e `tx.origin` per ogni frame.

Se tra qualche giorno ricordi sei idee

1

Una transazione avvia l'esecuzione, ma al suo interno possono esistere più call frame.

2

msg.sender è il chiamante diretto del frame corrente, non necessariamente l'origine della transazione.

3

msg.value e calldata sono input controllabili dall'esterno: si validano secondo i requisiti del protocollo.

4

storage è stato persistente: ogni scrittura deve preservare proprietà esplicite del sistema.

5

Un revert corretto non lascia una transizione parziale dello stato che si voleva atomica.

6

Parti da proprietà verificabili: cosa deve essere vero, e test anche per ciò che non deve essere possibile.

Da dove viene ogni affermazione

Fonte	Per che cosa
Solidity 0.8.37 Release Announcement	baseline del corso e relativi bugfix
Solidity · Units and Globally Available Variables	msg.sender, msg.value, msg.data, msg.sig, tx.origin
Solidity · Contract ABI Specification	function selector e codifica degli argomenti
Solidity · Expressions and Control Structures	require, revert, assert, custom errors
Solidity · Layout of State Variables in Storage	slot, packing, mapping e array dinamici
ethereum.org · EVM e Transactions	macchina a stati, transazioni, contesto
ethereum.org · Pectra EIP-7702 e EIP-7702	delegation, assunzioni su EOA e tx.origin
Foundry · config, Forge Std, cheatcodes, Cast	solc_version, vm.prank, vm.expectRevert, cast
OWASP SCSVS-AUTH-2 · SCWE-018 · SC05:2026	msg.sender per l'autorizzazione, tx.origin, input